

E-Commerce Sales Data Analysis Using SQL

I used **Microsoft SQL Server** to create and analyse an e-commerce sales database. The database consisted of multiple relational tables, including Customers, Products, Orders, OrderItems, and Payments tables to extract meaningful business insights.

I performed a join between **Orders and Customers** to link each order with customer-level information such as name and registration details. I then joined **Payments** with the Orders table to combine payment-related details, including payment method and amount. This combined dataset allowed me to analyse complete transaction-level information in a single view.

Using this unified data, I calculated the **total sales generated by each customer** and sorted the results in **descending order** to identify the **top 5 highest-spending customers**. I also computed the **Average Order Value (AOV)** to measure the typical revenue per order. Additionally, I analysed the **payment method breakdown** to understand which payment modes (UPI, Card, Wallet, etc.) were most frequently used by customers.

Overall, this SQL analysis provides clear insights into customer spending patterns, top-value customers, and payment preferences. These insights are useful for understanding business performance and supporting data-driven decision-making.

Database Schema:

- List of tables: Customers, Orders, Products, OrderItems, Payments

SQL Queries:

```
use master
```

```
go
```

```
create database Ecommerce
```

```
go
```

```
sp_helpdb 'Ecommerce'
```

```
go
```

use Ecommerce

go

--creating customer table

create table Customer_Details

(

customerID	INT	IDENTITY(1,1) PRIMARY KEY,,
------------	-----	-----------------------------

FirstName	VARCHAR(50),
-----------	--------------

LastName	VARCHAR(50),
----------	--------------

EmailAddress	VARCHAR(100),
--------------	---------------

RegistrationDate	DATE,
------------------	-------

Address	VARCHAR(150)
---------	--------------

)

go

--Inserting sample customer data

insert into Customer_Details(FirstName,LastName,EmailAddress,RegistrationDate,Address)

values

('Shiva', 'Kranthi', 'shiva.kranthi@example.com', '2024-01-05', 'Madhapur, Hyderabad'),

('Umesh', 'Reddy', 'umesh.reddy@example.com', '2024-02-10', 'Kukatpally, Hyderabad'),

('Kranthi', 'Kumar', 'kranti.kumar@example.com', '2024-03-12', 'Ameerpet, Hyderabad'),

('Sai', 'Kiran', 'sai.kiran@example.com', '2024-04-15', 'Gachibowli, Hyderabad'),

('Harshitha', 'Devi', 'harshitha.devi@example.com', '2024-05-20', 'Manikonda, Hyderabad'),

('Shivani', 'Rao', 'shivani.rao@example.com', '2024-06-18', 'Secunderabad, Hyderabad'),

```
('Sravanthi', 'Reddy', 'sravanthi.reddy@example.com', '2024-07-25', 'Kondapur, Hyderabad'),  
('Chiranjeevi', 'Varma', 'chiru.varma@example.com', '2024-08-04', 'Lingampally, Hyderabad'),  
('Charan', 'Teja', 'charan.teja@example.com', '2024-09-09', 'Banjara Hills, Hyderabad'),  
('Thirumal', 'Netha', 'thirumal.netha@example.com', '2024-10-14', 'Miyapur, Hyderabad'),  
('Anitha', 'Kumari', 'anitha.kumari@example.com', '2024-11-02', 'Hitech City, Hyderabad');
```

--To find the total number of customers

```
SELECT 'Customer_Details' AS TableName, COUNT(*) AS Rows FROM Customer_Details;
```

```
select*from Customer_Details
```

```
go
```

--Creating products table

```
CREATE TABLE Products (  
    ProductID          INT                      IDENTITY(1,1) PRIMARY KEY,  
    ProductName        VARCHAR(100),  
    Category           VARCHAR(50),  
    Price              DECIMAL(10,2),  
    Stock              INT  
)
```

```
go
```

```
INSERT INTO Products (ProductName, Category, Price, Stock)
```

```
VALUES
```

```
('Bluetooth Earbuds', 'Electronics', 1499.00, 120),
```

('Wireless Mouse', 'Electronics', 599.00, 200),
('Office Chair', 'Furniture', 3499.00, 50),
('Notebook Set', 'Stationery', 199.00, 500),
('Water Bottle', 'Lifestyle', 299.00, 300),
('Backpack', 'Lifestyle', 899.00, 150),
('Power Bank 10,000mAh', 'Electronics', 1299.00, 100),
('LED Table Lamp', 'Home Decor', 799.00, 80),
('Smart Watch', 'Electronics', 2499.00, 70),
('USB-C Cable', 'Accessories', 199.00, 400),
('Yoga Mat', 'Sports', 499.00, 90)

--To find the Total number of products

SELECT 'Products' AS TableName, COUNT(*) AS Rows FROM Products;

select*from Products

go

CREATE TABLE Orders (

OrderID	INT IDENTITY(1,1)	PRIMARY KEY,
CustomerID	INT	FOREIGN KEY REFERENCES Customer_Details (CustomerID),
OrderDate	DATE,	
TotalAmount	DECIMAL(10,2),	
Status	VARCHAR(20)	

)

go

```
INSERT INTO Orders (CustomerID, OrderDate, TotalAmount, Status)
```

```
VALUES
```

```
(1, '2024-01-10', 1499.00, 'Completed'),
```

```
(2, '2024-02-14', 599.00, 'Completed'),
```

```
(3, '2024-03-18', 3499.00, 'Completed'),
```

```
(4, '2024-04-20', 199.00, 'Completed'),
```

```
(5, '2024-05-28', 899.00, 'Completed'),
```

```
(6, '2024-06-22', 1299.00, 'Completed'),
```

```
(7, '2024-07-29', 799.00, 'Completed'),
```

```
(8, '2024-08-10', 2499.00, 'Completed'),
```

```
(9, '2024-09-16', 199.00, 'Completed'),
```

```
(10, '2024-10-19', 499.00, 'Completed'),
```

```
(11, '2024-11-05', 3499.00, 'Completed');
```

```
--TO find the total number of orderscount.
```

```
SELECT 'Orders' AS TableName, COUNT(*) AS Rows FROM Orders;
```

```
select*from Orders
```

go

```
CREATE TABLE OrderItems
```

```
(
```

```
OrderItemID
```

```
INT
```

```
IDENTITY(1,1) PRIMARY KEY,
```

OrderID	INT	FOREIGN KEY REFERENCES Orders(OrderID),
ProductID	INT	FOREIGN KEY REFERENCES Products(ProductID),
Quantity	INT,	
Price	DECIMAL(10,2)	

);

INSERT INTO OrderItems (OrderID, ProductID, Quantity, Price)

VALUES

(1, 1, 1, 1499.00),

(2, 2, 1, 599.00),

(3, 3, 1, 3499.00),

(4, 4, 1, 199.00),

(5, 6, 1, 899.00),

(6, 7, 1, 1299.00),

(7, 8, 1, 799.00),

(8, 9, 1, 2499.00),

(9, 10, 1, 199.00),

(10, 11, 1, 499.00),

(11, 3, 1, 3499.00);

--TO find the Total Orderitems count.

SELECT 'OrderItems'AS TableName, COUNT(*) AS Rows FROM OrderItems;

select*from OrderItems

go

CREATE TABLE Payments (

PaymentID	INT	IDENTITY(1,1) PRIMARY KEY,
OrderID	INT	FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
PaymentDate	DATE,	
PaymentMethod	VARCHAR(20),	
Amount	DECIMAL(10,2)	

);

INSERT INTO Payments (OrderID, PaymentDate, PaymentMethod, Amount)

VALUES

(1, '2024-01-10', 'UPI', 1499.00),
(2, '2024-02-14', 'Card', 599.00),
(3, '2024-03-18', 'UPI', 3499.00),
(4, '2024-04-20', 'Wallet', 199.00),
(5, '2024-05-28', 'Card', 899.00),
(6, '2024-06-22', 'UPI', 1299.00),
(7, '2024-07-29', 'UPI', 799.00),
(8, '2024-08-10', 'Card', 2499.00),
(9, '2024-09-16', 'UPI', 199.00),
(10, '2024-10-19', 'Wallet', 499.00),
(11, '2024-11-05', 'Card', 3499.00);

```
SELECT 'Payments' AS TableName, COUNT(*) AS Rows FROM Payments;
```

```
select*from Payments
```

```
go
```

```
--Orders with customer and payment info
```

```
SELECT o.OrderID, o.OrderDate, c.FirstName + ' ' + c.LastName AS Customer,
```

```
o.TotalAmount, o.Status,
```

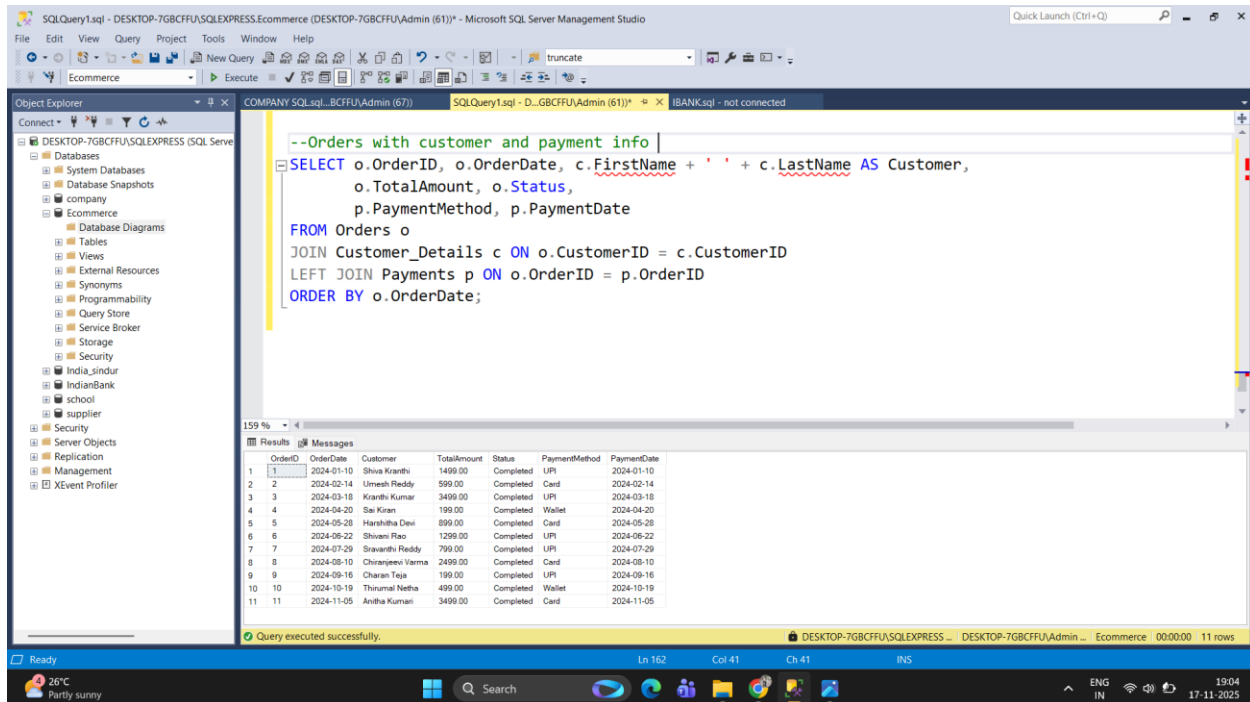
```
p.PaymentMethod, p.PaymentDate
```

```
FROM Orders o
```

```
JOIN Customer_Details c ON o.CustomerID = c.CustomerID
```

```
LEFT JOIN Payments p ON o.OrderID = p.OrderID
```

```
ORDER BY o.OrderDate;
```



The screenshot displays the Microsoft SQL Server Management Studio interface. The query editor shows the following SQL query:

```
--Orders with customer and payment info |
SELECT o.OrderID, o.OrderDate, c.FirstName + ' ' + c.LastName AS Customer,
o.TotalAmount, o.Status,
p.PaymentMethod, p.PaymentDate
FROM Orders o
JOIN Customer_Details c ON o.CustomerID = c.CustomerID
LEFT JOIN Payments p ON o.OrderID = p.OrderID
ORDER BY o.OrderDate;
```

The Results pane shows the output of the query, which is a table with 11 rows and 7 columns:

OrderID	OrderDate	Customer	TotalAmount	Status	PaymentMethod	PaymentDate
1	2024-01-10	Shiva Kranthi	1499.00	Completed	UPI	2024-01-10
2	2024-02-14	Umesh Reddy	599.00	Completed	Card	2024-02-14
3	2024-03-18	Kranthi Kumar	3499.00	Completed	UPI	2024-03-18
4	2024-04-20	Sai Kiran	199.00	Completed	Wallet	2024-04-20
5	2024-05-28	Harshitha Devi	899.00	Completed	Card	2024-05-28
6	2024-06-22	Shivan Rao	1299.00	Completed	UPI	2024-06-22
7	2024-07-29	Swarathi Reddy	799.00	Completed	UPI	2024-07-29
8	2024-08-10	Chiranjeevi Varma	2499.00	Completed	Card	2024-08-10
9	2024-09-16	Charan Teja	199.00	Completed	UPI	2024-09-16
10	2024-10-19	Thirumal Netha	499.00	Completed	Wallet	2024-10-19
11	2024-11-05	Anitha Kumari	3499.00	Completed	Card	2024-11-05

The status bar at the bottom indicates "Query executed successfully." and "11 rows".

--Total sales by customer (descending)

```
SELECT c.CustomerID, c.FirstName + ' ' + c.LastName AS Customer,  
  
       SUM(o.TotalAmount) AS TotalSales,  
  
       COUNT(o.OrderID) AS OrdersCount  
  
FROM Customer_Details c  
  
JOIN Orders o ON c.CustomerID = o.CustomerID  
  
GROUP BY c.CustomerID, c.FirstName, c.LastName  
  
ORDER BY TotalSales DESC;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The query editor contains the following SQL code:

```
--Total sales by customer (descending)  
  
SELECT c.CustomerID, c.FirstName + ' ' + c.LastName AS Customer,  
       SUM(o.TotalAmount) AS TotalSales,  
       COUNT(o.OrderID) AS OrdersCount  
FROM Customer_Details c  
JOIN Orders o ON c.CustomerID = o.CustomerID  
GROUP BY c.CustomerID, c.FirstName, c.LastName  
ORDER BY TotalSales DESC;
```

The Results pane shows the output of the query, which is a table with 4 columns: CustomerID, Customer, TotalSales, and OrdersCount. The data is sorted by TotalSales in descending order.

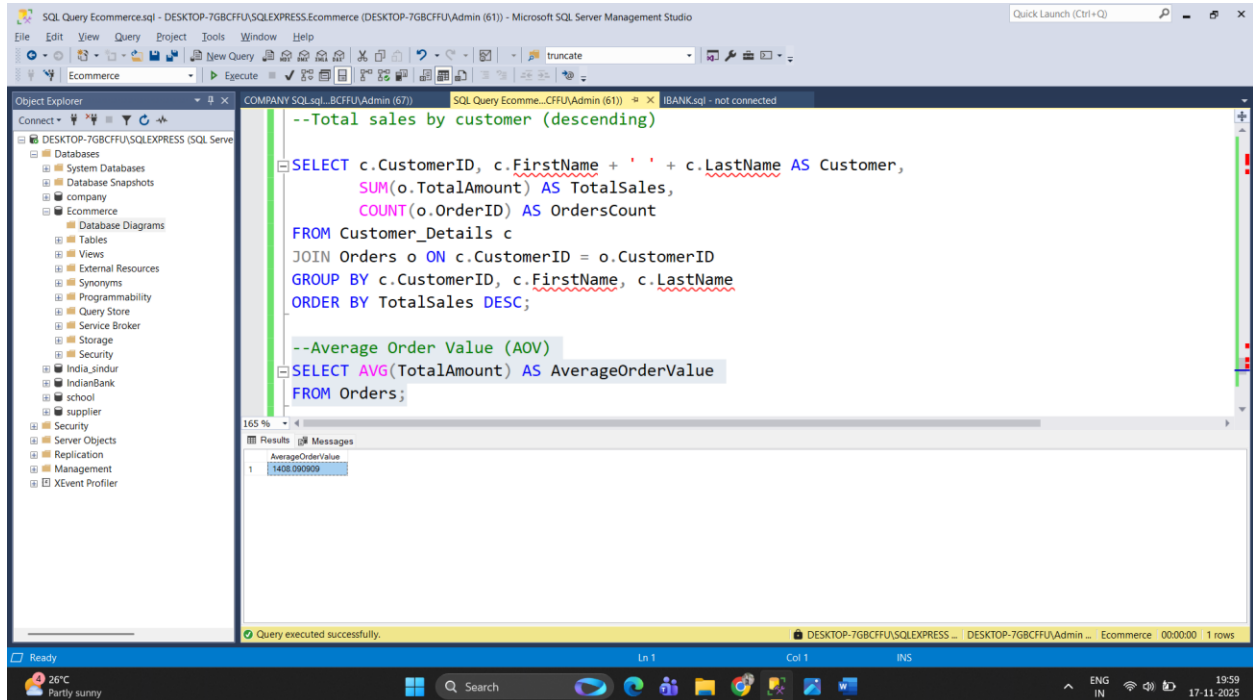
CustomerID	Customer	TotalSales	OrdersCount
3	Kranthi Kumar	3499.00	1
11	Anitha Kumari	3499.00	1
8	Chiranjeevi Varma	2499.00	1
1	Shiva Kranthi	1499.00	1
6	Shivani Rao	1299.00	1
5	Harshitha Devi	899.00	1
7	Sraavanthi Reddy	799.00	1
2	Umesh Reddy	599.00	1
10	Thirumal Neetha	499.00	1
4	Sai Kiran	199.00	1
9	Charan Teja	199.00	1

The status bar at the bottom indicates that the query was executed successfully, returning 11 rows in 00:00:00 seconds.

--Average Order Value (AOV)

SELECT AVG(TotalAmount) AS AverageOrderValue

FROM Orders;



--Payment method breakdown (counts and amounts)

SELECT PaymentMethod, COUNT(*) AS PaymentsCount, SUM(Amount) AS TotalAmount

FROM Payments

GROUP BY PaymentMethod

ORDER BY TotalAmount DESC;

SQLQuery1.sql - DESKTOP-7GBCFFU\SQLEXPRESS\Ecommerce (DESKTOP-7GBCFFU\Admin (61)) - Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Help

Object Explorer

- Connect
- DESKTOP-7GBCFFU\SQLEXPRESS (SQL Serve
- Databases
 - System Databases
 - Database Snapshots
 - company
 - Ecommerce
 - Database Diagrams
 - Tables
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - India_sindur
 - IndianBank
 - school
 - supplier
- Security
- Server Objects
 - Replication
 - Management
 - XEvent Profiler

SQLQuery1.sql - D:\GBCFFU\Admin (61)* - iBANK.sql - not connected

```
SELECT AVG(TotalAmount) AS AverageOrderValue
FROM Orders;

--Payment method breakdown (counts and amounts)
SELECT PaymentMethod, COUNT(*) AS PaymentsCount, SUM(Amount) AS TotalAmount
FROM Payments
GROUP BY PaymentMethod
ORDER BY TotalAmount DESC;
```

Results

	PaymentMethod	PaymentsCount	TotalAmount
1	Card	4	7496.00
2	UPI	5	7295.00
3	Wallet	2	698.00

Query executed successfully.

DESKTOP-7GBCFFU\SQLEXPRESS ... DESKTOP-7GBCFFU\Admin ... Ecommerce 00:00:00 3 rows

Ready 26°C Partly sunny 19:29 17-11-2025